Computer Isolation: A Security Review of Choir's VMs, vmctl and the Host Boundary

An owner's review, written to teach as well as to decide. 14 October 2026, about 17:20 to 19:00 UTC. Repository: `origin/main` at 9ee22b738. Live evidence: read-only inspection of Node B (choiros-b, kernel 6.18.16, Xeon E-2236) at about 17:20 to 17:45 UTC, with `sudo` used only to read `/proc/<pid>/status`, namespace links, `iptables -S`, `systemctl show`, `sysctl`, `ls -l`, and the CPU vulnerability files. Nothing on the host was changed. No secret value was read or printed; where this report names a secret it names the file, never its contents.

Owner (about 17:20 UTC): "we should do all the security work [...] I'm well aware that the VM manager, vmctl, hypervisor stuff is quite undercooked [...] Firecracker on root without the jailer, et cetera, that is a big oversight. And it's something that I should really understand better, not just fix it — where we are currently, where we need to go, what the different options and potentials are, how that may or may not vary using Cloud Hypervisor instead of Firecracker."

Mutation class of this document: **green** (docs only). Every proposal in it names its own class; most are **red** (vmctl, VM lifecycle, tenant isolation, host services).

How to read this review

Section 1 is a primer: what a hypervisor is, what "escaping a VM" means, and why the privileges of the process that runs the VM decide how bad an escape is. Section 2 is the threat model for Choir. Section 3 is where we are, layer by layer, with evidence. Section 4 is where we need to go. Section 5 compares Firecracker and Cloud Hypervisor through this lens. Section 6 surveys open-source jailers and sandboxes we can reuse instead of writing our own. Section 7 is the phased plan. Section 8 is what the red-team station LT should test first. Section 9 is a glossary.

Every finding carries a severity, written as **likelihood × impact**:

	meaning
likelihood **L / M / H**	how plausible the preconditions are for a motivated attacker in the next six months, given who can reach Choir today (mostly the owner and a few test accounts) and who will soon (strangers, model-written code, red-team agents)
impact **L / M / H / C**	what the attacker gets: one computer's own data (L/M), the host's network or platform services (H), every tenant plus the platform's keys (C, critical)
severity	the product, read as **Low / Medium / High / Critical**

Evidence labels: **live** (read on Node B today), **code** (read in the repository at the commit above, with a path), **doc** (an existing Choir document), **upstream** (a vendor document or advisory, cited), and **inference** (reasoning that has not been tested; treat as a hypothesis).

The short answer

1. **The single biggest risk is not a missing feature of Firecracker; it is that every Firecracker process runs with root's file and network identity inside the host.** Live on Node B: three VMMs, uid 0, capabilities `CAP_NET_ADMIN` and `CAP_SYS_PTRACE`, no chroot, vmctl's mount namespace (read-write over all of `/var/lib/go-choir`), the host's network namespace, the host's PID namespace, and one shared cgroup. Everything under `/var/lib/go-choir` is owned by root, so a root process reads it through ordinary owner permissions even without `CAP_DAC_OVERRIDE`: every computer's `data.img`, every `credential.img`, the provider credentials (`gateway-provider.env`, `codex-auth.json`), the auth service's JWT signing key, the escrow operator tokens, and the platform Dolt files. One guest-to-VMM escape that also gets past Firecracker's seccomp filter therefore becomes a compromise of every tenant and the platform. Severity: **High** today (L × C), rising to Critical when strangers run code.

2. **"Loopback" is a credential on Node B, and every host process holds it.** vmctl's internal API (stop, remove, hibernate, list any computer), the gateway's credential routes, corpusd's and maild's internal routes, and both Dolt servers (root, no password) all trust any caller on 127.0.0.1. The VMMs, the SearXNG meta-search service that parses untrusted web pages, the Go module proxy, and Qdrant all live in that network namespace. A foothold in any of them is platform control. Severity: **High** (M × H).
3. **Host services run as root with full capabilities.** auth, proxy, gateway, corpusd, maild and both Dolt servers have `User=` empty and the full capability bounding set; their shared `SystemCallFilter` re-adds the `@privileged` set it was meant to remove; the proxy can write the auth signing-key directory. Severity: **Medium-High**.
4. **Guest egress is open to the internet.** The S1 security floor's "default-deny VM egress" is not built: guests MASQUERADE to anywhere, and can reach host sshd. Severity: **Medium** (H × M): exfiltration of a computer's own data and abuse from Node B's address.
5. **Side channels are unmitigated for strangers.** Node B's Coffee Lake CPU reports L1TF and MDS "SMT vulnerable", SMT is on, core scheduling is unused, and Firecracker has not opted in to the per-process speculation barriers. Severity: **Low** while all tenants are the owner's, **High** the day a stranger's computer shares a core with yours.
6. **The tap firewall fix is ready** (`fix/tap-deny-guest-range`, 816077e83): a default-deny guest forwarding chain over the whole address pool, source-checked INPUT rules, teardown on every exit, and an orphan sweep. It is not deployed. One gap remains in it: inbound forwarding to a guest (`FORWARD -o <tap> -j ACCEPT`) is still unconditional.
7. **Firecracker versus Cloud Hypervisor changes the device-model surface, not the confinement problem.** Neither VMM is safe to run as root in the host's namespaces. Firecracker ships a jailer that we do not use; Cloud Hypervisor ships seccomp and Landlock but no jailer. Both need the same per-VM confinement contract, which Choir should own once and apply to both (section 4.2, section 6).

First three actions (section 7 has the full plan):

1. **Deploy the tap firewall fix** with one added rule (inbound forwarding only for established flows), and kill the stray root `python3 -m http.server 19090 --bind 0.0.0.0` that has been serving `/tmp` since 30 September. (Today; red; small.)
2. **Slice 1 of the VMM privilege drop:** run each Firecracker under the jailer with its own uid/gid, chroot, mount and PID namespaces and a cgroup v2 leaf; drop `CAP_SYS_PTRACE` from vmctl; opt the VMM in to speculation barriers. Prove it with `/proc/<pid>/status` and an assumed-breach probe. (This week; red; medium.)
3. **Put a token on loopback:** authenticate vmctl, gateway credential, corpusd and maild internal routes with a per-service credential (a unix socket with `SO_PEERCRED`, or a bearer from a root-only file), and give Dolt a password; then start moving host services to their own users. (This week to this month; red; medium.)

1. Primer: hypervisors, escapes, jailers and side channels

1.1 What a VMM is, and what KVM does

A virtual machine is a program that believes it owns a computer. Two pieces make that illusion on Linux:

- **KVM** is part of the Linux kernel. It uses the CPU's virtualization extensions (Intel VT-x, AMD-V) to run the guest's instructions directly on the real CPU, at nearly full speed, in a hardware mode where the guest cannot touch host memory or host devices. When the guest does something the hardware will not let it do alone (touch a device register, take certain faults), the CPU stops the guest and returns control to KVM: a **VM exit**. KVM is reached through the `/dev/kvm` device file.
- The **VMM** (virtual machine monitor) is an ordinary user-space process that opens `/dev/kvm`, allocates the guest's memory, and **emulates the devices** the guest sees: the network card, the disks, the serial console, the memory balloon. Firecracker and Cloud Hypervisor are VMMs. When the guest writes to its virtual network card, the CPU exits to KVM, KVM hands the event to the VMM, and the VMM's Rust code reads the guest's request out of guest memory and turns it into a `write()` on a host tap device. The guest's memory is mapped into the VMM's address space, so the VMM can read and write all of it.

Choir uses virtio devices: a standard where the guest places requests in shared-memory rings ("virtqueues") and the VMM processes them. That processing code, in the VMM, is the main place where guest-controlled data meets host code.

1.2 The guest/host boundary, and what "escaping the VM" means

The guest kernel runs as "root inside the VM". Nothing the guest's root does can directly affect the host: the CPU enforces that. To affect the host, an attacker who already has root in a guest has to find a bug in one of three places:

1. **The VMM's device emulation** (most common in practice): a malformed virtio request that makes the VMM's code write out of bounds, use freed memory, or follow a host path it should not. Firecracker's CVE-2026-5747 (virtio-PCI queue size changed after activation, out-of-bounds write) and Cloud Hypervisor's CVE-2026-45782 (virtio-blk use-after-free) and CVE-2026-27211 (a guest writes a QCOW2 header into a raw image and the VMM then follows the header's backing-file path on the host) are this class. The result is **code execution inside the VMM process**, with the VMM's privileges.
2. **KVM itself** (rarer, more severe): a bug in the kernel's handling of VM exits or instruction emulation. The result is **code execution in the host kernel**, and no user-space confinement helps.
3. **The CPU** (side channels): the guest cannot run code on the host, but it can *read* data the host or another guest left in shared CPU structures (section 1.5).

"Escaping the VM" usually means case 1. The important consequence for us: **after a case-1 escape, the attacker is whatever the VMM process is.** If the VMM is an unprivileged user in an empty chroot with no network and its own cgroup, the attacker has gained very little: they hold their own VM's memory and disk, which they had already. If the VMM is root in the host's namespaces, they have the host.

1.3 Why the VMM's privileges decide the blast radius

Security engineers call this **defence in depth** or "assume breach": design each layer as if the one in front of it has already failed. The VMM is the layer most likely to fail (it parses attacker-controlled data at high speed), so it is the layer that must hold the least authority. The question to ask of any VMM process is: *if this process were the attacker, what could it read, write, signal, connect to, or exhaust?* The answers come from:

property	where it is visible	what it limits
user and group (uid/gid)	`/proc/<pid>/status` `Uid:` `Gid:`	which files it can open by ordinary permissions; which processes it can signal
capabilities	`CapEff:` `CapBnd:`	root's special powers, split into ~40 pieces (`CAP_NET_ADMIN` reconfigures networking and firewalls; `CAP_SYS_PTRACE` attaches to and reads other processes; `CAP_DAC_OVERRIDE` ignores file permissions)
root directory	`readlink /proc/<pid>/root`	which part of the filesystem exists at all for it (a chroot or pivot_root)
mount namespace	`/proc/<pid>/ns/mnt`	which mounts it sees
network namespace	`/proc/<pid>/ns/net`	which interfaces and which loopback (127.0.0.1) it shares
PID namespace	`/proc/<pid>/ns/pid`	which other processes it can see
cgroup	`/proc/<pid>/cgroup`	how much CPU, memory, I/O and how many tasks it may use before the kernel throttles or kills it
seccomp filter	`Seccomp:` `Seccomp_filters:`	which system calls it may make at all
no-new-privileges	`NoNewPrivs:`	whether it can ever gain privileges by executing a setuid program

Seccomp deserves a note. It is a kernel filter, installed by the process on itself, that cannot be removed afterwards. Firecracker installs one filter per thread type (API thread, VMM thread, vCPU threads) that allows only the system calls that thread needs. That is strong: an attacker with code execution in a vCPU thread cannot, for example, call `execve`. But it is not a wall on its own. All threads share one address space, so an attacker who controls one thread can corrupt the memory of another thread whose filter allows more (inference; this is a general property of per-thread filters, not a known Firecracker exploit). And seccomp says nothing about *which files* a permitted `open` may reach: that is the job of uid, chroot and mount namespace. Seccomp narrows the verbs; the jailer narrows the nouns.

1.4 What the Firecracker jailer does

The jailer is a small, separate Rust program shipped with Firecracker. You run the jailer as root; it builds a cage, then executes Firecracker inside it as an unprivileged user. Its documented steps (upstream `docs/jailer.md`):

1. **Per-VM directory.** It creates `<chroot_base>/<exec_file_name>/<id>/root/` and copies the Firecracker binary into it (a copy, not a hard link, so two VMMs never share code pages). Whatever the VM needs (kernel, root filesystem, data image, snapshot files) must be hard-linked or bind-mounted into that directory by the caller before launch.
2. **Namespaces.** It enters a new mount namespace, optionally joins a **network namespace** you created (`--netns /var/run/netns/<id>`), and optionally creates a new **PID namespace** (`--new-pid-ns`).
3. **Chroot.** It uses `pivot_root` into the per-VM directory, so `/` for the VMM is that directory and nothing else of the host exists.
4. **Device nodes.** It creates only `/dev/kvm`, `/dev/net/tun`, `/dev/urandom` and (if present on the host) `/dev/userfaultfd` inside the chroot, owned by the VM's uid/gid.
5. **Cgroups.** It places the process in its own cgroup (v1 by default; pass `--cgroup-version 2`, `--cgroup cpu.weight=...`, `--cgroup memory.max=...`, `--parent-cgroup`), so one VM cannot starve the host.
6. **Resource limits.** It sets `rlimit`s (`--resource-limit fsize=...,no-file=...`).
7. **Privilege drop.** It closes every inherited file descriptor above 2, clears the environment, switches to the given **uid and gid** (`--uid`, `--gid`; a non-root uid holds no capabilities after exec), and `exec`s Firecracker, which then sets no-new-privileges and installs its own **seccomp** filters. The jailer itself installs no seccomp filter and no Landlock rules.

After that, a Firecracker escape lands as, say, uid 30117 in a chroot that contains one VM's files, a network namespace that contains one tap, a PID namespace that contains one process, and a cgroup that caps its memory. That is the difference between "one tenant" and "every tenant".

1.5 Side channels: SMT, L1TF, MDS and the Spectre family

A side channel leaks data without breaking any permission check, by measuring timing in hardware that two programs share. They matter for VMs because they cross the guest/host boundary without any software bug.

- **SMT** (simultaneous multithreading, Intel "Hyper-Threading") lets one physical core run two hardware threads at the same instant, sharing the core's L1 cache, fill buffers and branch predictors. If a stranger's vCPU runs on one thread and yours on the sibling, they share that state.
- **L1TF** (L1 Terminal Fault, "Foreshadow", 2018) and **MDS** (Microarchitectural Data Sampling: RIDL, Fallout, ZombieLoad, 2019) are Intel bugs that let code on one hyperthread read data its sibling recently touched in the L1 cache or CPU buffers, **across VM boundaries**. The kernel mitigates the same-thread case by flushing caches and buffers on VM entry and exit, but it **cannot** mitigate the sibling case while SMT is on: the kernel says "SMT vulnerable". Node B reports exactly that.
- **Spectre v1/v2 and relatives** (2018 onwards) trick the CPU's branch prediction into speculatively running code that reads secrets and leaves traces in the cache. Mitigations are barriers the kernel or a process asks for: **IBPB** (Indirect Branch Prediction Barrier: flush predictor state when switching between distrusting contexts), **STIBP** (stop sibling threads from steering each other's predictions), **SSBD** (disable speculative store bypass). On current kernels most of these are "conditional": a process gets them only if it opts in with `prctl` or runs under seccomp with the right kernel defaults.

- **Core scheduling** (Linux 5.14+) is the middle path between SMT on and SMT off: processes carry a "cookie", and the scheduler only lets two threads share a physical core at the same moment if their cookies match. One cookie per tenant means your vCPUs never run beside a stranger's.
- **Memory deduplication (KSM)** merges identical pages across VMs, which lets one guest time whether another guest has a given page (Flush+Reload). Firecracker's production guide says to turn it off.

1.6 The picture

```
ATTACKER-CONTROLLED INPUT
web pages, model output, capsule code, a stranger's tenant
      |
      v
+------------------------ guest computer (KVM VM) -------------------------+
|  Yaegi cell   import allowlist; not a security boundary (I18)            |
|  capsule      user/pid/mount/net namespaces, cgroup, seccomp,            |
|               Landlock, L2 egress broker                                 |
|  guest OS     NixOS; runtime as guest root today; signer users           |
+--------------------------------------------------------------------------+
      |  virtio rings, MMIO/PCI exits         <-- boundary A: hardware
      v
+----------------------- VMM process: firecracker -------------------------+
|  device emulation in Rust; per-thread seccomp                            |
|  TODAY:  uid 0, CAP_NET_ADMIN + CAP_SYS_PTRACE, host netns,              |
|          vmctl's mount ns (rw /var/lib/go-choir), shared cgroup          |
|  TARGET: per-VM uid, chroot, own mount/pid/net ns, own cgroup, no caps   |
+--------------------------------------------------------------------------+
      |  syscalls                             <-- boundary B: process
      v
+------------------------ host kernel (Linux 6.18, KVM) -------------------+
|  side channels: SMT siblings, caches, predictors  <-- boundary C: CPU    |
+--------------------------------------------------------------------------+
      |  loopback, files, signals             <-- boundary D: services
      v
+----------------------- host services (all root today) -------------------+
|  vmctl :8083, proxy :8082, gateway :8084, corpusd :8086, maild :8087,    |
|  auth :8081, Dolt :13306/:13307, caddy, searx, goproxy, qdrant           |
|  keys: auth JWT key, platform signing key, escrow key,                   |
|        provider credentials, escrow operator tokens                      |
+--------------------------------------------------------------------------+
      |  SSH (root), GitHub Actions deploy key, provider APIs
      v
+------------------------ other hosts and the world -----------------------+
+--------------------------------------------------------------------------+
```

Each arrow is a trust boundary. A good design makes each one hold on its own; today boundaries B and D are thin, so A (the hardware plus Firecracker's code quality) carries almost all of the weight.

2. Threat model for Choir

2.1 Assets

asset	where it lives	why it matters
Owner and user data images	`/var/lib/go-choir/vm-state/<vm>/data.img` (ext4, sparse, 32 GiB nominal, mode 0644 in a 0750 root directory, live); quarantined copies `data.img.quarantine-*` kept beside them	everything a computer has stored: documents, Dolt workspace, source trees, the guest's signer keys and privacy key under `/mnt/persistent`
Credential disks	`<vm>/credential.img` (8 MiB ext4, 0600) and `<vm>/persist/gateway-token` (0600)	the computer's gateway bearer and its event envelope
Snapshots	`<vm>/snapshot/{vmstate,mem,meta.json}` (0600 in a 0700 directory), unencrypted, kept up to 36 h, deleted after a healthy resume (code: `internal/vmmanager/snapshot_lifecycle.go:162-208`, `snapshot.go:526-540`)	`mem` is the guest's whole RAM: in-memory tokens, the privacy data key, signer keys once loaded, open documents
The tape and its projections	platform Dolt (Store A, `:13306`) and corpus Dolt (Store B, `:13307`), files under `/var/lib/go-choir/platform-dolt`, `corpus-dolt`	event heads, route slots, the published object graph: integrity matters more than secrecy

asset	where it lives	why it matters
Signing and escrow keys	auth JWT key `/var/lib/go-choir/auth-signing/ed25519-key`; platform signing key and escrow key under `/var/lib/go-choir/platform-artifacts/`; escrow operator tokens `secrets/corpusd-escrow-operators.env` (all root, 0600 or in 0700/0750 directories, live)	the JWT key mints any user's session; the escrow key unwraps every computer's escrowed data key
Provider credentials	`gateway-provider.env`, `codex-auth.json`, `maild.env`, `searxng.env` (root 0600, live)	spend and reputation with model and mail providers; guests never hold them (code: `nix/autoputer-vm.nix:15,740`)
Platform database credentials	none: Dolt accepts `root` with no password on loopback (code: `VMCTL_ROUTE_DSN`, `nix/node-b.nix`)	write access to Store A is route and head authority
Other tenants	every other computer on the host	the multi-tenant promise itself
The host's network identity	Node B's public address	abuse from it damages mail deliverability, provider accounts, and hosting standing

2.2 Attackers and entry points

attacker	entry point	first boundary they meet	realistic goal
Malicious web content read by research	host-side fetches (SearXNG on the host; gateway search), guest-side readers	the parser in whichever process reads it; prompt injection into the desks	steer a desk into running code or leaking data; exploit a parser
Model output executed in Yaegi cells	every desk turn	the Yaegi import allowlist, then the capsule (I18: "Yaegi is not the security boundary")	ambient authority in the guest runtime
Capsule code built by engineering	`go build`, `go test`, arbitrary Bash in an effects-capable assignment	capsule namespaces, seccomp, Landlock, cgroup, egress broker	guest root; then the VMM
A malicious tenant (future strangers)	full root inside their own computer, by design	the VMM device model (boundary A) and the CPU (boundary C)	other tenants' data; the platform's keys
A compromised guest daemon	a bug in the runtime, signer or updater	guest users and systemd confinement; then host services over the tap	the computer's own keys; host internal routes
A compromised host service	a bug in SearXNG, the Go module proxy, Caddy, the proxy, the gateway	the service's own systemd sandbox; then loopback	platform control (section 3.5)
Supply chain	Go modules through the capsule proxy; npm, PyPI and crates directly from capsules; nixpkgs; GitHub Actions (tag-pinned) running with a root deploy key	Nix hashes and go.sum for builds; nothing for the Actions runner's root SSH	code execution in a capsule, a build, or as root on Node B
The hosting provider or an on-link neighbour	physical disks, the provider's L2 segment	disk encryption (none today); host forwarding rules	read disks at rest; reach guests through Node B's forwarding

2.3 Trust boundaries, innermost first

1. **Cell → capsule.** A Yaegi cell is model-written Go interpreted inside a worker. The import allowlist removes `os`, `os/exec`, `net`, `net/http` and friends. Doctrine I18 is explicit that this is an attenuated vocabulary, not a sandbox.
2. **Capsule → guest OS.** The capsule is the real sandbox for model-written processes: Linux namespaces, cgroup v2, seccomp, Landlock and a broker that mediates effects and egress.
3. **Guest OS → VMM.** The hardware boundary (KVM) plus the VMM's device emulation. A malicious tenant starts here: they are root in their guest by design.
4. **VMM → host kernel.** The VMM's process confinement: uid, capabilities, namespaces, chroot, cgroup, seccomp. This is the layer the jailer builds, and the thinnest one today.
5. **Host kernel → host services.** Loopback, files, signals and `/proc` between processes on the host. Today a shared root identity and unauthenticated loopback.

6. **Host services → other hosts.** SSH, CI, provider APIs, backups, the SP standby and the future EPYC.

A defensible design makes each boundary hold even if the one inside it has failed. The rest of this review checks each in turn.

3. Where we are now, layer by layer

A summary first, then the evidence. Severity is likelihood × impact as defined above.

#	finding	layer	severity	fix ready?
F1	VMMs run as root, `CAP_NET_ADMIN` + `CAP_SYS_PTRACE`, no chroot, vmctl's mount view (rw over all Choir state), host net and PID namespaces, one shared cgroup	VMM	**High** → Critical with strangers	no (section 4.2)
F2	Loopback is the credential: vmctl, gateway credential routes, corpusd, maild internal routes and passwordless Dolt trust every host process	host services	**High**	no
F3	Host services run as root with full capabilities; `SystemCallFilter` re-adds `@privileged`; proxy can write the JWT signing-key directory	host services	**Medium-High**	no
F4	Management, Texture and research cell workers run as unconfined guest root; the Yaegi allowlist is their only boundary	cell	**Medium-High**	no
F5	Guest runtime is unconfined root (S1 remainder); `pdftotext` and Obscura parse untrusted input as root; Obscura has no address guard	guest OS	**Medium**	no
F6	Guest egress open to the internet (S1 "default-deny egress" not built); guests reach host sshd	network	**Medium**	no
F7	Tap firewall: latent /16 gap, INPUT ordering, fail-open policy, 157 stale rules	network	**Medium** (latent)	**yes**, branch `fix/tap-deny-guest-range`
F8	Inbound forwarding to taps unconditional (`-o tap -j ACCEPT`), also on the fix branch	network	**Medium** until tested	one rule to add
F9	Gateway bearers bound to "some guest", not to their VM (except Jev); 720 h lifetime	identity	**Medium**	partly (Jev pin pattern)
F10	maild takes the alert recipient from a guest header; escrow operator routes have no transport check	host services	**Medium** / **Low-Medium**	no
F11	SMT side channels (L1TF, MDS "SMT vulnerable"), no core scheduling, VMM not opted in to IBPB/SSBD, KSM on (idle)	CPU	**Low** now → **High** with strangers	no
F12	Snapshots hold guest RAM in plaintext on an unencrypted disk; data images 0644; no disk encryption on Node B	at rest	**Medium**	no
F13	CI deploys as root over SSH with a tag-pinned Actions supply chain; stray root HTTP server on :19090; dev toolchains on the production host	host access	**Medium**	partly (kill the server)

3.1 Cells: Yaegi workers (boundary 1)

Strong. The import allowlist is enforced twice: the interpreter is built with only filtered standard-library symbols (`unsafe`, `reflect`, `runtime`, `syscall`, `os`, `os/exec`, `net`, `net/http` and `plugin` have no symbols to bind; Yaegi's unrestricted maps are never loaded), and each cell's imports are parsed and refused before it runs (`internal/yaegikernel/allowlist.go:10–42`, `session.go:90–91`, `eval.go:83–121`). The worker gets a minimal environment with no token (`desk_session_worker.go:144,174–195`). `choir.Exec` and `WriteFile` are exported only to engineering (`choir.go:334,443–453`).

Weak.

- **Management, Texture and research workers are plain children of the guest runtime**, re-executed from the runtime binary, with only `Setpgid` and `Pdeathsig` set (`internal/yaegikernel/desk_worker_spawn_linux.go:27–33`) and an 8 GiB `RLIMIT_AS`. No user change (so **guest root, full capabilities**), no seccomp, no Landlock, no namespaces, no cgroup. A Yaegi interpreter bug (it is not

documented as a sandbox, and its `choir` exports are built from reflection) gives the attacker guest root: the gateway token, the privacy key, open egress, and the host's guest-facing ports. Doctrine I18 already says Yaegi is not the security boundary; for three of four desks it currently is.
- The management and Texture session handles still *authorize* `ActionExec` and `ActionWriteFile` (`choir.go:129`, default branch); only the missing export blocks them. One wrong export would open exec.
- `ReadFile` and `ListDir` confine paths lexically (`broker.go:456–466`) and follow symlinks while running as root (inference: exploitable only if a symlink can be planted under the working root).

Severity of F4: **Medium-High** (M × M-H: the likelihood is "a bug in an interpreter fed adversarial code every turn"; the impact is one computer's full authority plus a launch point at boundary 3).

3.2 Capsules and the guest OS (boundaries 2 and 3)

Strong: capsules are a real sandbox. Engineering's capsule (code: `internal/capsule`, `cmd/capsule-broker`) is the best-built boundary in the system:

- a **user namespace** mapping capsule root to guest uid 65534 with `setgroups` denied, plus mount, network, UTS, IPC, cgroup and PID namespaces and a chroot into an overlay (`executor.go:34,507–518`, `capsule-broker/main.go:308`);
- **capabilities** cut to `DAC_OVERRIDE` + `FOWNER` for workloads, inside the user namespace only (`capabilities.go:59–85`);
- a **deny-by-default seccomp** filter with `NoNewPrivs` and TSYNC; no `mount`, `unshare`, `setns`, `ptrace`, `bpf`, `perf_event_open`, `keyctl`, `io_uring_*`, `userfaultfd`, `kexec` (`seccomp.go:14–75`);
- **Landlock** V5, mandatory, fail-closed (`landlock.go:31–126`);
- **cgroup v2**: `memory.max` 1 GiB, no swap, `cpu.max` 1 CPU, `pids.max` 256;
- `/run`, `/mnt`, `/proc`, `/sys` masked; `/proc` remounted `hidepid=2`; environment allowlisted with `TOKEN`/`KEY`/`CHOIR_` names stripped (`capsule-broker/main.go:212–220,477–505`);
- **egress**: non-network capsules get an empty network namespace; network ("L2") capsules get only loopback, with `127.0.0.1:3128` forwarded over a unix socket to an allowlisting proxy in the runtime: CONNECT to 443 only, a host suffix allowlist (package registries, GitHub, Hugging Face, arXiv and similar), guest-side DNS with private-address refusal, budgets and per-connection logs (`egress.go:103–117`); Go modules through the host's read-through proxy (`internal/goproxy`).

Weak, capsules.

- `clone` and `clone3` are allowed without flag filtering, so a capsule can create nested user namespaces (seccomp cannot inspect `clone3` flags). User namespaces are the most common doorway to kernel bugs (overlayfs-in-userns, netfilter). Severity: **Low-Medium**.
- Every capsule shares guest uid 65534; capsules are separated from each other only by chroot, mount namespace and Landlock. Fine while an assignment has one capsule; revisit when N run at once.
- The egress allowlist includes `github.com`, `api.github.com` and Hugging Face, which accept uploads: a 16 MiB-per-capsule exfiltration channel the proxy cannot inspect (TLS). Severity: **Low-Medium** (it is the capsule's own computer's data).
- The capsule's seccomp allows unfiltered `ioctl` and `prctl`, `memfd_create` and `vmsplice` (historically involved in kernel exploits). Severity: **Low**; tighten opportunistically.

Weak, guest OS (F5). The guest runtime unit `go-choir-autoputer` (`nix/autoputer-vm.nix:743–853`) has no `User=`, `NoNewPrivileges`, `CapabilityBoundingSet`, `SystemCallFilter` or `ProtectSystem`; only `ReadWritePaths` (inert without `ProtectSystem`) and `InaccessiblePaths` over the signer keys and updater control. Status in `docs/problems/s1-non-root-runtime-child-uid-boundary-2026-10-04.md`: "partially repaired; non-root runtime + child-UID boundary open". Inside that root runtime, untrusted content is parsed by `pdftotext` (unsandboxed, `internal/content/pdf_blocks.go:20`, `extract.go:237`) and by Obscura for Web Lens, also as root, whose navigate path checks only the URL scheme (`internal/browsercontrol/handler.go:80–94,261–275`; SSRF to the tap host and loopback is likely, inference). The guest kernel carries no hardening sysctls

(`unprivileged_bpf_disabled`, `yama.ptrace_scope`, `kptr_restrict` all unset). The signer and updater units, by contrast, are hardened and run as their own users. Severity: **Medium**: everything a cell escape or a PDF parser bug reaches is guest root.

3.3 Hardware boundary: KVM and Firecracker's device model (boundary A)

Strong.

- KVM hardware virtualization on a host kernel that is current (6.18.16) with `nx_huge_pages=Y` (iTLB multihit mitigated) and PTI on (live).
- Firecracker's device model is deliberately tiny: virtio-net, virtio-blk, balloon, serial, a partial i8042 and RTC. No virtio-fs, no VFIO, no USB, no graphics, no vsock in Choir's config (code: `internal/vmmanager/manager.go:1707–1722`). Every device that does not exist is an escape that cannot happen.
- Firecracker's per-thread seccomp filters are on: `Seccomp: 2`, `Seccomp_filters: 6`, and no `--no-seccomp` anywhere (live; code).
- Firecracker is pinned at **1.15.1**, the patched release for CVE-2026-5747 (virtio-PCI out-of-bounds write affecting 1.13.0 to 1.14.3 and 1.15.0). Choir launches with `--enable-pci`, so this pin matters: never roll back below 1.15.1 while PCI is on (doc: isolation-runtimes §1).
- The drives are minimal: a read-only erofs Nix store, the computer's own `data.img`, the credential disk and nothing else.

Weak.

- **The newest, least mature part of Firecracker is the one Choir turns on.** `--enable-pci` puts the virtio-PCI transport (added in 1.13) in front of every device; its first high CVE was this year. Severity: **Low** as a standalone finding (L × C), but it is the most likely first link in an escape chain. The 1.17.0 upgrade (density lever and isolation research item) carries further fixes.
- **Nested virtualization is enabled on the host** (`kvm_intel nested=Y`, live). Firecracker does not expose VMX to guests, so this is not a path today; it becomes one for any backend or CPU template that does (Cloud Hypervisor exposes nested virtualization on x86 when the host allows it). Set `nested=0` unless a backend needs it. Severity: **Low**.
- `/dev/kvm` is mode 0666 (live; NixOS default). Any local user can create VMs and reach KVM's code. Under a jailer the VMM gets its own device node; the host-wide node can then go to `0660 root:kvm`.

3.4 The VMM process (boundary B): the central finding

Live, Node B, 17:24 UTC (three VMMs: pids 1317715, 1317861, 1318688):

property	value	what it means
`Uid` / `Gid`	`0 0 0 0`	root
`CapEff` / `CapBnd`	`0x81000` = `CAP_NET_ADMIN` + `CAP_SYS_PTRACE`	can reconfigure every interface and firewall rule on the host; can attach to processes
`NoNewPrivs`	1	cannot gain more by exec'ing setuid binaries (good)
`Seccomp`	2, 6 filters	Firecracker's own filters active (good)
root (`/proc/<pid>/root`)	`/`	no chroot
mount namespace	`4026532751`, the same as vmctl's, not the init namespace (`4026531832`)	vmctl's systemd sandbox: `ProtectSystem=strict` (host read-only) **but** `ReadWritePaths=/var/lib/go-choir` (everything Choir owns writable) and `ReadOnlyPaths=/var/lib/go-choir/auth`
network namespace	`4026531833` = init	the host's own interfaces and loopback
PID namespace	`4026531836` = init	sees and can signal every root process on the host (uid 0)
cgroup	`0::/system.slice/go-choir-vmctl.service` for all three	no per-VM CPU, memory or I/O bound; one VM can starve the others and vmctl

property	value	what it means
`Speculation_Store_Bypass`	`thread vulnerable`	not opted in to SSBD
`SpeculationIndirectBranch`	`conditional enabled`	not opted in to IBPB/STIBP
`Cpus_allowed_list`	`0-11`	any core, any SMT sibling
environment	vmctl's whole environment, including `VMCTL_ROUTE_DSN` (a passwordless root Dolt login)	code: `startFirecrackerProcess`, `manager.go:1942-2003` (no `cmd.Env`, no `SysProcAttr`)

What uid 0 reaches without `CAP_DAC_OVERRIDE`. Losing `CAP_DAC_OVERRIDE` sounds like it should stop root from reading other people's files. It does not help here, because there are no other people: every file under `/var/lib/go-choir` is owned by root (live `ls -l`), so a uid-0 process passes the **owner** permission check on all of them. Within the VMM's mount view that includes:

- every computer's `data.img` (35 computer directories, live) and the quarantined copies;
- every `credential.img` and `persist/gateway-token`;
- every snapshot directory when computers hibernate;
- `gateway-provider.env`, `codex-auth.json`, `maild.env`, `searxng.env`;
- `auth-signing/ed25519-key` (the JWT signing key; directory is in the read-write set);
- `secrets/corpusd-escrow-operators.env`;
- `platform-artifacts/` (platform signing key, escrow key) and both Dolt data directories;
- vmctl's `ownerships.json` (the tap-IP to computer map that every guest identity check relies on), writable.

What `CAP_NET_ADMIN` adds. In the host's network namespace it can flush or rewrite iptables (removing every tap isolation rule), add routes, re-address taps, and sniff or inject on any tap. **What `CAP_SYS_PTRACE` adds.** Read and write the memory of any process on the host (the gateway holds provider keys in memory; corpusd holds the escrow key). Neither capability is needed by the VMM. vmctl needs `CAP_NET_ADMIN` for `ip` and `iptables` (code: `manager.go:3089-3338`); no code shows a need for `CAP_SYS_PTRACE` (it came with commit 7a15266d7, 2026-04-11; the only candidate consumer is opening `/proc/<pid>/fd/1` of a VMM, which root can do to its own children without it — inference).

The API socket is a root file primitive. Node B runs in snapshot mode, so each VMM serves `api/fc.sock` (directory 0700, root). Its `/snapshot/create` and `/snapshot/load` take host paths; with a root VMM and no chroot, anyone who can reach the socket can make Firecracker write or read any path in its mount view (inference from the API; root-only today, so it only matters after another compromise).

Exploit preconditions. (1) Root in some guest: free for a malicious tenant, plausible for capsule code that escapes the capsule. (2) A device-model bug giving code execution in the VMM: a 0-day; the virtio-PCI transport and virtio-blk are the likeliest. (3) Getting from "code in a seccomp-filtered thread" to useful host effects: either a filter that permits `openat`/`read`/`write` on host paths in some thread, a second thread corrupted through shared memory, or a kernel bug reachable through permitted syscalls. With a jailer, step (3) still lands in an empty chroot as an unprivileged user. Without it, step (3) lands on everything above.

Severity: High (likelihood L today: no known exploit, small surface; impact C: every tenant, every platform key, provider spend). It becomes **Critical** when strangers have root in their own guests, because step (1) is then free. Doc: `docs/problems/firecracker-runs-as-root-without-jailer-2026-10-14.md`.

3.5 Host services and loopback (boundary D)

Live listeners (`ss -ltnp`, 17:30 UTC):

port	process	bind	reachable from guests?
22	sshd	0.0.0.0, ::	**yes**: NixOS firewall allows 22 on every interface, including taps (live `nixos-fw`)

port	process	bind	reachable from guests?
80, 443	caddy	*	yes (public anyway)
2019	caddy admin API	127.0.0.1	no (but only the INPUT port list stands between a guest and it; see route_localnet)
8081	auth	127.0.0.1	no
8082	proxy	*	yes, DNAT from tap peer
8083	vmctl	127.0.0.1	yes, DNAT from tap peer to 127.0.0.1
8084	gateway	*	yes
8086	corpusd	*	yes
8087	maild	*	yes
8088	goproxy	127.0.0.1	yes, DNAT
13306, 13307	Dolt (platform, corpus)	127.0.0.1	no
6333, 6334	qdrant	127.0.0.1	no
8888	searxng	127.0.0.1	no
19090	**`python3 -m http.server 19090 --bind 0.0.0.0`, root, cwd `/tmp`, started 30 September from an SSH session**	0.0.0.0	no (firewall); but any local process can read the host's `/tmp` through it

Weak: loopback is the credential.

- **vmctl** (`internal/vmctl/handlers.go:1237–1255`, `isInternalCaller`) admits loopback, a unix socket, **or any `192.0.2.*` address** (a test convenience in production code). Its internal routes stop, remove, hibernate, hold, list and relay to any computer (`handlers.go:1261–1288`); `/internal/debug/pprof/` is open to host callers. No token, no mTLS.
- **gateway** credential issue/ensure routes need loopback plus the `X-Internal-Caller` header (`internal/gateway/handlers.go:705–710`); the ensure route also imports an unknown `computerID:token` as a new active identity (`config.go:381–398`).
- **corpusd** internal routes: loopback plus the header (`internal/platform/authority.go:21–63`).
- **maild** internal routes: loopback plus the header (`internal/maild/api.go:205–215`).
- **Dolt**: `root@tcp(127.0.0.1:13306)` with no password (code: `nix/node-b.nix`, `VMCTL_ROUTE_DSN`; dolt `sql-server` started without a config user, live). Any local process can read and write Store A and B: event heads, route slots, the object graph.

So the question "who is internal?" has the answer "every process in the host network namespace": the three VMMs, SearXNG (which fetches and parses untrusted web pages), the Go module proxy (which downloads and serves untrusted module archives), Qdrant, Caddy, and the stray Python server. The `X-Internal-Caller` header adds nothing against a local attacker: anyone can send it. Severity: **High** (M × H): a single RCE in SearXNG or goproxy becomes the ability to destroy computers, mint gateway identities, and rewrite the tape's heads.

Weak: host services run as root with full capabilities (live `systemctl show`): auth, proxy, gateway, corpusd, maild, both Dolt servers, checkpointd and front-importance have `User=` empty and the full capability bounding set (`cap_chown cap_dac_override cap_dac_read_search cap_fowner ...`). Only goproxy, qdrant and searx use `DynamicUser`. The shared `commonServiceHardening` block (`nix/node-b.nix:154–178`) is good in intent (`ProtectSystem=strict`, `PrivateTmp`, `PrivateDevices`, `NoNewPrivileges`) but its `SystemCallFilter = [ "@system-service" "~@mount" "@privileged" ]` **adds `@privileged` back**: live, the allowed list includes `bpf`, `chroot`, `acct` and `add_key`. The proxy has **read-write** access to `auth-signing/` (`node-b.nix:414–415`) although it needs only the public key; the gateway has read-write over all of `/var/lib/go-choir`. No unit sets `IPAddressDeny`, `RestrictAddressFamilies` (except goproxy and searx) or `ProtectHome`. Severity: **Medium-High**.

Weak: the guest trusts "the host" as one principal. Inside the guest, `server.HostSourcedCaller` (`internal/server/host_caller.go:28–56`) accepts the host peer address (the tap's `.1`), **loopback**, TEST-NET-1 and unix sockets as "host-sourced", and on that basis honours `X-Authenticated-User` and the `/internal/runtime/*` routes. Two consequences:

- *On the host side*, any host process can open a TCP connection to `10.200.x.2` (OUTPUT is unfiltered, the source address is the tap's `.1`) and be "the host" to that guest, including asserting any user header. Severity: **Medium** (needs a host foothold; then it is one more lateral path).
- *On the guest side*, any process on the guest's own loopback (the terminal, a capsule child if it shares the guest's network namespace, the third-party `zot` agent) can assert the owner header to the runtime. The stricter `HostPeerCaller` (`host_caller.go:65–98`) exists but is used only for diagnostics. Severity: **Low-Medium** (identity is the guest's own owner; the risk is bypassing per-route checks).

Weak: tokens are bound to transport, not to the VM. The gateway checks the bearer and that the peer is *some* Choir guest address (`internal/gateway/handlers.go:643–664, isAuthorizedRuntimePeer`); apart from the Jev route, a bearer leaked from computer A works from computer B or from the host. Bearers live 720 h with no rotation (`node-b.nix, GATEWAY_AUTOPUTER_TOKEN_TTL`). The Jev re-pin fix (2cc8ab8fa, today) is correct in shape: the pin moves only through host lifecycle control (`EnsureCredentialForPeer, internal/gateway/config.go:336–399`), never from the guest. Generalizing that pin to every gateway route is the obvious next step. Severity: **Medium** (M × M: provider spend under another computer's identity).

Weak: guest-reachable routes that do not check the transport.

- **Escrow operator routes** (`internal/platform/key_escrow_http.go:297–312, authorizeKeyEscrowOperator`) check only `X-Choir-Operator` and a token hash; corpusd is reachable from every guest. A leaked operator token would work from a guest. Tokens are random and compared in constant time, so this is defence in depth, not an open door. Severity: **Low-Medium**.
- **maild** takes `X-Authenticated-Email` from the request even for a guest caller (`internal/maild/api.go:216,222`); it becomes the recipient of risk-alert mail (`notifications.go:29–65`). A guest can make Choir's mail domain send alerts to any address. Severity: **Medium** for abuse and deliverability.
- The proxy does not strip client-supplied `X-Resolved-Autoputer-URL` and `X-Proxy-Trusted-*` on ingress; they are overwritten in production paths (`internal/proxy/handlers.go:258–300`), so this is latent. Severity: **Low**.

Weak: `route_localnet=1` on every tap (live; code `manager.go:3259`). It is what makes the DNAT from the tap peer to 127.0.0.1 work, and it also means a guest can send packets addressed to 127.0.0.1 directly. Today the per-tap INPUT port list and the NixOS firewall stop them at the same ports the DNAT allows, so there is no extra exposure; but it puts Caddy's admin API (127.0.0.1:2019, which can rewrite all routing) and both Dolt servers one mistaken INPUT rule away from every guest (inference). The cleaner shape is to bind the guest-facing services on the tap side (or a dedicated `choir-guest` address) and drop `route_localnet`.

3.6 Host network and the tap firewall

Live (17:28 UTC): policies `INPUT ACCEPT, FORWARD ACCEPT, OUTPUT ACCEPT`; `net.ipv4.ip_forward=1`; IPv6 forwarding off; three live taps (`10.200.115.1/30, .116, .117`). FORWARD holds 135 rules: per live tap a destination deny for `10.200.0.0/16` and a source anti-spoof deny (both with 0 hits), then 122 stale ACCEPTs for vanished `vm-vm-<hash>-tap` interfaces, then per live tap `-i tap -j ACCEPT` and `-o tap -j ACCEPT`. 2,417 packets have fallen through to the ACCEPT policy since the counters were last reset; nobody has classified them.

Strong. Per-tap anti-spoofing (S1a), the destination deny for the first /16, NixOS's reverse-path filter in `mangle` (drops forged sources whose return path is not the same tap), and the S1a refusal matrix proven on staging on 4 October (`docs/evidence/s1a-refusal-matrix-2026-10-04.json`).

Weak.

- **Latent cross-guest gap**: the deny covers 1 of the 16 /16s the allocator uses. Severity: **Medium** (M × H) the day an allocation reaches 10.201; the old scheme reached 10.202. Fix ready.
- **INPUT ordering**: the tap-to-host service ACCEPTs sit ahead of the anti-spoof DROP. Severity: **Low** (blind spoofing within the tap's own /30). Fix ready.
- **Fail-open**: FORWARD policy ACCEPT; a tap whose rules were never installed, or were lost, forwards anywhere. Teardown misses crashes and failed spawns; 157 stale rules prove it. Fix ready.
- **Inbound forwarding to guests is unconditional**: `FORWARD -o <tap> -j ACCEPT` admits any forwarded packet whose route leads to a tap, not only replies. Host-originated traffic to a guest goes through OUTPUT, not FORWARD, so FORWARD only needs to admit replies to guest-initiated flows (`-m conntrack --ctstate ESTABLISHED,RELATED`). With `ip_forward=1`, a host on Node B's provider segment that routes `10.200.0.0/13` to Node B's MAC address may reach guest ports directly (inference; untested; whether the provider's network delivers such frames is unknown). **The fix branch keeps this rule unchanged** (`hostfirewall.go:140` on the branch). Severity: **Medium** (L × H) until tested.
- **Egress is open**: `POSTROUTING -s <guest>/30 ! -d <guest>/30 -j MASQUERADE` plus `-i tap -j ACCEPT` means any destination on the internet, any protocol. The S1 acceptance item "VM egress is default-deny rather than merely masqueraded" is not met; the S1a slice deliberately left egress open pending a recording proxy. Inside the guest, capsules have their own egress allowlist (section 3.1), but the guest runtime, the desks' cells and anything with guest root do not. Severity: **Medium** (H × M): a compromised guest exfiltrates its own computer's data and uses Node B's address for abuse.

3.7 The CPU (boundary C): side channels

Live (`/sys/devices/system/cpu/vulnerabilities`, 17:26 UTC; Xeon E-2236, microcode 0xfa):

item	Node B reports	meaning
`l1tf`	PTE Inversion; VMX: conditional cache flushes, **SMT vulnerable**	a guest on one hyperthread can read L1 data of its sibling
`mds`, `mmio_stale_data`	Clear CPU buffers; **SMT vulnerable**	same, for CPU fill and store buffers
`spectre_v2`	IBRS; IBPB: conditional; STIBP: conditional	barriers only for processes that opt in
`spec_store_bypass`	disabled via prctl	SSBD only for processes that opt in
`vmscape`	IBPB before exit to userspace	the 2025 guest-to-host-userspace branch issue is covered
`retbleed`, `meltdown`, `srbds`, `gather_data_sampling`, `tsx_async_abort`	mitigated	
`smt/control`	on	
`kvm_intel vmentry_l1d_flush`	cond	L1 flushed on VM entry only after "interesting" exits
KSM	`run=1`, `pages_sharing=0`	on but merging nothing (Firecracker never marks memory mergeable)
core scheduling	`CONFIG_SCHED_CORE=y`, unused	doc: computer-cpu-efficiency §5
VMM speculation state	`Speculation_Store_Bypass: thread vulnerable`, `SpeculationIndirectBranch: conditional enabled`	Firecracker has not opted in

Today's exposure is small because nearly every computer belongs to the owner. A side channel between your own computers leaks nothing you did not already have. The exposure becomes real the day a stranger's computer runs on Node B: two tenants' vCPUs on sibling threads is a live L1TF/MDS channel on this CPU, and the kernel cannot close it while SMT is on. Severity of F11: **Low** today, **High** with strangers (H × H: the attacks are public, need no bug in Choir, and read whatever the victim's vCPU touched, including keys).

Three cheaper items do not depend on tenancy and should be fixed with the jailer slice: opt the VMM in to IBPB/STIBP and SSBD with `prctl` (lever C13 in the CPU report), turn KSM off (`run=0`) so a future change cannot silently enable cross-tenant deduplication, and keep the shared-pmem lever (L10) behind a security panel as the density report already says.

3.8 Data at rest: snapshots, images and disks

Strong. Snapshot files are 0600 inside a 0700 partial directory and renamed into place atomically (`snapshot_lifecycle.go:162–208`); they are deleted after a healthy resume, on fallback, on budget eviction and by an orphan sweep, with a 36-hour age limit (`snapshot.go:526–540,659,758–801`). Snapshot metadata strips the gateway token and envelope (`snapshot.go:323–332`). The guest-side privacy key and signer keys live in 0700/0400 locations. Escrow wrapping uses X25519 ECDH, HKDF-SHA256 and XChaCha20-Poly1305 with the computer ID bound in, and reveal needs two approvers (`internal/keyescrow/keyescrow.go`, `key_escrow_http.go:363–371`).

Weak (F12).

- **A snapshot's `mem` file is the guest's RAM in plaintext**: the gateway bearer the runtime re-reads from its credential disk, the privacy data key, signer keys once loaded, open documents. It sits on disk for up to 36 hours (inference on contents; code on lifetime).
- **Node B's disks are not encrypted**: btrfs on md RAID1 over two NVMe drives, no LUKS (live `lsblk`). Every data image, snapshot, key file and Dolt store is plaintext to anyone with the drives: the hosting provider, a replaced drive that leaves the data centre, or a future backup that is copied off the host.
- `data.img` files are mode 0644 (code: `manager.go:2732–2757`; live). The 0750 root directory above them blocks non-root users today; the mode should still be 0600 so that a future group or ACL change does not expose every disk.
- Quarantined data images (`data.img.quarantine-*`, 32 GiB nominal each) are kept beside live ones with no stated retention.
- The escrow key acts as its own KEK with no HSM or KMS (deferred by design, `keyescrow.go:1–12`), so whoever reads `platform-artifacts/` holds every escrowed computer key.

Severity: **Medium** (L × H). Cloud Hypervisor's snapshot offload daemon (v53) can stream a snapshot through an external encryptor; Firecracker needs the encryption done after the file is written, or a UFFD page server that encrypts. Full-disk encryption is the cheaper first step.

3.9 Host access, CI and supply chain (boundary 6)

Live.

- sshd: port 22 on all addresses, `PermitRootLogin prohibit-password`, password and keyboard-interactive login off, `AllowTcpForwarding yes`. Only root has a login shell. 2,038 failed or invalid-user attempts in the last 24 hours (internet background noise; keys only, so not a risk by itself). Root's authorized keys are the operator's and `github-actions-deploy` (code: `nix/node-b.nix:196–210`); the deploy key logged in from many cloud addresses in the last day.
- **CI is root on Node B.** A compromised workflow, a compromised third-party Action (they are referenced by tag, not commit SHA), or a stolen deploy key gives root on the host. That is the shortest path from "someone else's mistake" to "every tenant".
- **The stray server**: `python3 -m http.server 19090 --bind 0.0.0.0`, root, working directory `/tmp`, started 30 September from an SSH session. Blocked from outside by the NixOS firewall, but it serves the host's `/tmp` to every local process. Kill it.
- `nodejs`, `go`, `gcc` and `strace` are installed system-wide on the production host (code), a convenience for an intruder.

- No kernel lockdown (`/sys/kernel/security/lockdown` empty), modules not disabled, `kernel.io_uring_disabled=0`, unprivileged user namespaces allowed (`user.max_user_namespaces=127328`), `kernel.yama.ptrace_scope=1`, `unprivileged_bpf_disabled=2`, `lsm=landlock,yama,bpf` (no AppArmor or SELinux). These are defaults; each is a hardening knob for the host.

Strong. Nix pins builds (`vendorHash`, `npmDepsHash`, `flake.lock`); the Go module proxy re-verifies `go.sum` and limits redirects to proxy.golang.org and storage.googleapis.com; guests never hold provider keys. Severity of F13: **Medium** (L × C for CI; the server is Low).

4. Where we need to go

4.1 Principles

1. **Assume each layer is breached; size the next layer for that.** The VMM is the layer most likely to fail, so it holds the least.
2. **One isolation contract, many runtimes.** Write down what a confined VMM may touch (section 4.2) once, as a `Confinement` struct in the backend seam the isolation-runtimes report proposed, and make every backend (Firecracker, Cloud Hypervisor, later a desktop runtime) satisfy it. Verify the contract from outside (`/proc/<pid>/status`), not from the code that set it.
3. **Privilege lives in small programs.** A short privileged helper does the few things that need root; everything else, including the controller, runs unprivileged.
4. **Authority is a credential, never a location.** "Came from loopback" and "came from a guest address" are hints. Routes that change state need a token, a peer credential on a unix socket, or a key.
5. **Fail closed and sweep.** Default-deny chains, teardown on every exit path, an orphan sweep at start.

4.2 The VMM: a per-VM cage (the jailer contract)

This is the target for every VMM process, written concretely for Choir on Node B. Firecracker's jailer provides almost all of it; for Cloud Hypervisor the same contract is built from the engines surveyed in section 6.

property	target	how (Firecracker)
uid/gid	one per VM, from a reserved range, e.g. `uid = gid = 900000 + ordinal` where `ordinal` is the host-port ordinal vmctl already allocates (0 to 4095, `manager.go:371–373`); recorded in the ownership; files chowned on allocation; never reused while files of a previous holder remain	jailer `--uid --gid`
filesystem	a per-VM root containing only that VM's files: `/var/lib/go-choir/jail/firecracker/<vm-id>/root/` with `firecracker` (copied by the jailer), `vmlinux` and `initrd` (hard links or read-only bind mounts of the Nix store files), `store.erofs` (read-only), `data.img` and `credential.img` (owned by the VM uid, 0600), `snapshot/` (VM uid, 0700), `run/fc.sock`, and nothing else	jailer `--chroot-base-dir /var/lib/go-choir/jail`; vmctl places files before launch. Keeping the jail under `/var/lib/go-choir` keeps it on the same btrfs subvolume so `data.img` can be a hard link (no copy, no mount to clean up)
mount namespace	private; pivot_root into the jail; `/proc` not mounted unless needed	jailer default
PID namespace	its own; the VMM is PID 1 of a namespace that contains only itself	jailer `--new-pid-ns`
network namespace	its own, holding only that VM's tap (slice 2); until then, the host's, with an iptables `OUTPUT -m owner --uid-owner 900000-904095 -j DROP` rule so VMM uids cannot open any IP connection (they need none: tap I/O is a file descriptor and the API socket is a unix socket)	jailer `--netns /var/run/netns/choir-<vm-id>`
devices	only `/dev/kvm`, `/dev/net/tun` (and `/dev/userfaultfd` if UFFD restore lands), created in the jail for the VM uid; host-wide `/dev/kvm` back to `0660 root:kvm`	jailer creates them

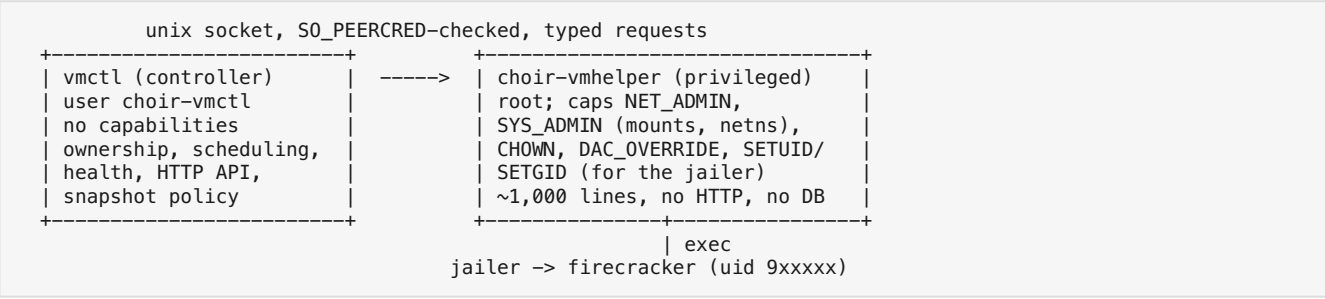
property	target	how (Firecracker)
cgroup v2	a leaf per VM under a delegated slice, e.g. `/sys/fs/cgroup/choir-vms.slice/<vm-id>`, with `cpu.weight` (the fractional-core lever), `memory.max` = guest memory + VMM overhead + snapshot page-cache allowance (measure first; start at guest + 25%), `memory.high` below it, `memory.swap.max=0`, `io.weight` and an `io.max` ceiling on the data disk, `pids.max=64`	jailer `--cgroup-version 2 --parent-cgroup choir-vms.slice --cgroup cpu.weight=... --cgroup memory.max=...`; the slice comes from systemd with `Delegate=yes`
capabilities	none (a non-root uid after exec holds none); `NoNewPrivs=1`	jailer drops to the uid before exec
seccomp	Firecracker's default per-thread filters, never `--no-seccomp`; later a custom filter (`--seccomp-filter`) that removes file creation from the API thread in computers that never snapshot	Firecracker
rlimits	`no-file` 2048; `fsize` large enough for the data image and the memory file, no larger	jailer `--resource-limit`
speculation	`PR_SET_SPECULATION_CTRL` for indirect branch (IBPB/STIBP) and store bypass (SSBD) set on the launching thread before exec, inherited by the VMM; a core-scheduling cookie per tenant (`PR_SCHED_CORE`) once strangers share a host	vmctl helper, before `exec(jailer)`
API socket	inside the jail (`run/fc.sock`), owned by the VM uid; vmctl's helper reaches it by path (needs `CAP_DAC_OVERRIDE` in the helper only, or a group the helper belongs to)	Firecracker `--api-sock /run/fc.sock` (path relative to the jail)
environment	empty except what Firecracker needs (nothing secret)	`cmd.Env = []string{}`
verification	`/proc/<pid>/status`: `Uid` in range, `CapEff 0`, `NoNewPrivs 1`, `Seccomp 2`; `readlink /proc/<pid>/root` = the jail; `ns/mnt`, `ns/pid` (and later `ns/net`) differ from init and from each other; `/proc/<pid>/cgroup` = its own leaf	an acceptance probe (section 7)

What happens to snapshot and restore. Firecracker writes the snapshot to paths *relative to the jail*, so vmctl passes `snapshot/vmstate` and `snapshot/mem`, and reads them through the host path `<jail>/root/snapshot/`. Restore needs the snapshot files in the jail of the VM being restored, owned by its uid, which is the same jail when the computer keeps its uid. Firecracker's snapshot identity already pins the launch config hash (S3 failure mode 9); a change of paths inside the config invalidates snapshots once, at the cutover. Plan it as a cold-boot window or a planned-restart marker (CLAUDE.md restart rule). **Balloon** and free page reporting are unaffected. **virtio-pmem and virtio-mem** (density levers) need their backing files in the jail too; a shared read-only pmem image works as a read-only hard link but is exactly the Flush+Reload channel the density report flagged.

What vmctl must stop doing. Reattach and reaping match `/proc/*/cmdline` for a basename containing `firecracker` (`manager.go:2163–2220`). Under the jailer, the process is `firecracker` with jailer-chosen arguments, and pids are in another namespace. Track VMMs by a pidfd and by cgroup membership (`cgroup.procs` of the VM's leaf) instead; that is also backend-neutral.

4.3 vmctl: a privilege split

Today vmctl is root with `CAP_NET_ADMIN` and `CAP_SYS_PTRACE` and does everything. The target is two programs:



The helper's whole vocabulary: `PrepareJail(vm)`, `CreateTap(vm)` / `DeleteTap(vm)`, `ApplyFirewall(vm)` / `RemoveFirewall(vm)`, `SweepOrphans()`, `Launch(vm, spec)`, `Signal(vm, sig)`, `CollectSnapshot(vm)`. Every argument is a VM ID that the helper maps to paths and names itself; it never accepts a path or an iptables argument

from the controller. The branch's `hostfirewall.go` is already shaped like this (a narrow interface over `iptables` with a fake for tests). Precedents: libvirt (privileged `virtqemud` launches QEMU as `qemu:qemu` under SELinux/AppArmor), and Firecracker's own split (the jailer is the privileged part; the VMM is not).

Two steps toward it that are small and safe on their own: drop `CAP_SYS_PTRACE` from vmctl's bounding set (no code needs it; verify the console-adoption path on staging), and give the helper's eventual operations their own Go package with no HTTP imports, so the split is a process boundary later rather than a rewrite.

4.4 Host services: least privilege and authenticated loopback

- **Users.** Each host service gets its own user (`DynamicUser=yes` with `StateDirectory=` where its state is private; a static user where state is shared). Remove the default capability set (`CapabilityBoundingSet=` empty) everywhere except where a capability is named and justified.
- **Fix the filter.** `SystemCallFilter = [ "@system-service" "~@privileged" "~@mount" ]` (or drop the third entry): today's list re-adds `@privileged`.
- **Narrow file access.** Split the auth public key into its own directory so the proxy needs read-only access to that alone; the gateway gets `ReadWritePaths` over its own files, not all of `/var/lib/go-choir`; secrets move to systemd `LoadCredential=` so they appear only in the service's private `/run/credentials/<unit>/`.
- **Authenticate loopback.** Move vmctl's internal API to a unix socket (it already supports one: `VMCTL_AUTOPUTER_PROXY_SOCK`) with mode 0660 and a group per client class, check `SO_PEERCRED` uid against an allowlist per route, and keep the TCP listener only for the guest-scoped ComputerVersion routes. Do the same for the gateway's credential routes, corpusd's and maild's internal routes, or give each internal caller a bearer delivered with `LoadCredential=`. Give Dolt a password (or a unix socket) delivered the same way. Remove the `192.0.2.*` test allowance from production builds and turn pprof off unless a flag enables it.
- **Network policy per service.** `IPAddressDeny=any` plus `IPAddressAllow=localhost` (and the tap range where guests call in) on services that never talk to the internet; `RestrictAddressFamilies` on all of them. SearXNG and goproxy, which fetch untrusted content, should not be able to reach other services' loopback ports at all: run each in its own network namespace (`PrivateNetwork=yes` plus a proxy socket) or filter with `iptables -m owner --uid-owner`.

4.5 Network

- **Deploy the fix branch** (CHOIR-GUEST-FWD default-deny over the whole pool, INPUT source checks, teardown on every exit, orphan sweep).
- **Add one rule to it:** inbound forwarding to a tap only for established flows: `-o vm-+ -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`, then DROP. Host-to-guest connections go through OUTPUT and are unaffected.
- **Egress default-deny for the whole guest** (the S1 acceptance item): a host-side forward proxy per computer with an allowlist, logs and budgets, the same policy shape as the capsule's L2 broker; until then, at least drop SMTP (25), the provider's metadata and private ranges, and the host itself except the service ports.
- **Close sshd to guests**: `INPUT -i vm-+ -p tcp --dport 22 -j DROP` (the NixOS firewall allows 22 on every interface).
- **Retire `route_localnet`**: bind guest-facing listeners on a dedicated guest-side address (or the tap peer) instead of DNAT to 127.0.0.1.
- **Per-VM network namespace** (slice 2 of the jailer work): the tap lives in the VM's namespace, linked to the host by a veth pair whose host end carries today's per-tap rules. A VMM escape then sees no host interface and no host loopback.

4.6 Identity

- **Bind every bearer to its computer's current address**, the Jev pin generalized: the gateway accepts a bearer only from the guest IP the registry currently assigns to that computer; vmctl moves the pin on each boot through the loopback-only ensure route it already uses.

- **Make "the host" one principal.** The proxy signs the identity headers it injects (an HMAC with a per-computer key from the credential disk); the guest verifies the signature instead of trusting the source address. Then a stray host process, or the guest's own loopback, cannot assert the owner. Use `HostPeerCaller` (not `HostSourcedCaller`) for user routes in the meantime.
- **Longer term: vsock or mTLS.** A virtio-vsock connection's peer CID is set by the VMM, not the guest, so it cannot be spoofed from inside the guest: an identity channel that does not depend on firewall rules. A per-computer client certificate on the credential disk is the network-native alternative.
- **Shorter bearer lifetimes and rotation** on each boot.

4.7 CPU side channels: a policy per host

host	tenants	policy
Node B (Coffee Lake: L1TF, MDS)	owner only (today)	SMT on; VMM speculation opt-in; KSM off; `kvm_intel nested=0`
Node B	strangers	core scheduling with **one cookie per tenant** (owner ID), set by the helper at launch; or `nosmt` on hosts that take strangers; `vmentry_l1d_flush=always`
EPYC 7371 (Zen 1: no L1TF/MDS; Spectre v2, SSB, Retbleed, SRSO)	strangers	core scheduling per tenant, STIBP on, SRSO mitigation on; SMT can stay on (CPU report §5)
any	a "dedicated" tier	whole cores (and on the EPYC whole CCXs) pinned with `cpuset`, siblings idle

Measure the cost (`core_sched.force_idle_usec` per VM cgroup, throughput and p99) before choosing; the CPU report estimates 5 to 15% for core scheduling and 25 to 35% for SMT off. Cloud Hypervisor can set a per-VM core-scheduling cookie itself (v52); Firecracker needs the helper to do it.

4.8 Data at rest

- **Full-disk encryption** (LUKS) on the EPYC at bring-up and on Node B at its next rebuild, with a key that is not on the same disk (a network unlock such as Tang/Clevis, or a TPM).
- **Snapshots on an ephemeral-key volume.** Put snapshot directories on a dm-crypt volume whose key is generated at each host boot and never written down. A planned hibernate and wake works as today; a host reboot makes every hibernated computer cold-boot, which loses no durable data (the data image persists) and agrees with the crash rule. This removes "guest RAM on disk in plaintext" with almost no code.
- `data.img` 0600 and owned by the VM uid; a retention rule for quarantined images.
- An HSM or KMS-backed KEK for the escrow key when SP brings a second host.

4.9 Cells, capsules and the guest

- **Put the three other desks' cell workers in a capsule-lite**: the same user namespace, seccomp, Landlock and cgroup code the engineering capsule uses, without a writable overlay or network. Their egress already goes through the runtime's broker frames, so they need no network namespace contents at all.
- **Finish S1**: a non-root guest runtime with a full systemd sandbox; terminal children under their own uid; guest sysctls (`unprivileged_bpf_disabled=1`, `yama.ptrace_scope=2`, `kptr_restrict=2`, `dmesg_restrict=1`).
- **Sandbox the parsers**: `pdftotext` and Obscura in a capsule-lite; give Obscura's navigate the same address guard `sourcefetch` uses.
- **Tighten the capsule filter**: return `ENOSYS` for `clone3` (glibc falls back to `clone`, whose flags seccomp can inspect) and deny `CLONE_NEWUSER` in `clone`; this is how Docker's default profile handles it.
- **gVisor as an option for capsules**: `runsc` with the systrap platform runs inside a VM without nested virtualization and puts a user-space kernel between capsule code and the guest kernel. It is the strongest available answer to "capsule escape via a guest kernel bug"; its cost on Go builds is unmeasured. Make it an LT comparison (T4), not a default.

5. Firecracker and Cloud Hypervisor through this lens

The isolation-runtimes research (docs/research/isolation-runtimes-2026-10-14.md) covers features, density and migration. This section looks only at isolation.

5.1 The confinement story

	Firecracker 1.15.1 (1.17.0 upstream)	Cloud Hypervisor v54
official jailer	yes, `jailer` (Rust): uid/gid, pivot_root, mount/PID namespaces, joins a netns, cgroup v1/v2, rlimits, closes fds. Scoped to Firecracker (it prepends `--id` and timing arguments)	**none**; the docs show `setcap cap_net_admin` and nothing about uid, chroot or cgroups
seccomp	per-thread filters on by default; custom filters with `--seccomp-filter`	per-thread filters on by default; `--seccomp true
file self-confinement	none in the VMM	**Landlock** (`--landlock`, `--landlock-rules`, since v41): the VMM restricts its own file access to declared paths at VM creation, so paths for later hotplug must be declared up front
networking without privilege	opens a named tap; needs the tap in its netns and openable by its uid	takes a pre-created tap by name **or by file descriptor** (`--net fd=...`), so the VMM needs no `CAP_NET_ADMIN` if vmctl creates the tap already up with MAC and MTU set (`host_mac=` and `mtu=` still need the capability); a v54 fix makes restore work without `CAP_NET_ADMIN`
SMT side channels	nothing in the VMM; host core scheduling by the launcher	`--cpus core_scheduling=vm` (default since v52): vCPU threads of one VM share a cookie. **Per VM, not per tenant**; per-tenant cookies still need the launcher
upstream threat model	guest is untrusted; the jailer and host setup guide are part of the model	guest is untrusted. The management API, **snapshot files and migration peers are trusted input**. The VMM is not hardened against guest-driven CPU or memory exhaustion ("use control groups"). It does not use user namespaces, AF_ALG or `splice`, so an outer filter can block them

What changes with Cloud Hypervisor. It brings two inner layers that Firecracker lacks: Landlock, and fd-passed taps that remove the need for a networking capability. It brings no outer layer. Running it the way Choir runs Firecracker today (root, host namespaces, no cgroup) would be strictly worse, because the device model behind that missing outer layer is larger.

5.2 Device-model attack surface

surface	Firecracker	Cloud Hypervisor	what it means for Choir
devices	virtio-net, -blk, -balloon, -pmem, -mem, -rng, serial, partial i8042, RTC; virtio-PCI opt-in (Choir uses it)	everything Firecracker has, plus ACPI, CPU/memory/device hotplug, vhost-user, virtio-fs (through virtiofsd), vDPA, TPM, UEFI, VFIO	more devices means more parser code facing the guest. Turn off what Choir does not use
2026 advisories	CVE-2026-5747 (high, virtio-PCI OOB write, fixed 1.15.1); CVE-2026-1386 (medium, **jailer** symlink follow, fixed 1.13.2/1.14.1)	CVE-2026-45782 (high, virtio-blk async I/O use-after-free, fixed v52.0/v51.2); CVE-2026-27211 (high, guest writes a QCOW2 header into a raw image and format autodetection then follows a host backing file, fixed v51.0/v50.1)	both projects patch quickly. The CH QCOW2 bug is exactly the class a chroot neutralizes: the host path it followed would not exist in a jail. Always set `image_type=raw`
virtio-fs	absent	via a separate `virtiofsd` process per share	it exposes a host directory tree to the guest. It is a new escape class (Kata's CVE-2026-47243 was guest root to host root through virtiofs). If used, run virtiofsd under its own uid, sandbox and read-only share. **Recommendation: do not use it for tenant data**; keep block devices

surface	Firecracker	Cloud Hypervisor	what it means for Choir
VFIO / GPU passthrough	absent	present	it hands a physical device, with DMA, to the guest. It needs an IOMMU, interrupt remapping and per-device isolation groups, plus firmware trust. Treat GPU hosts as a separate tenancy class (one tenant per device, devices reset between tenants)
live migration	absent	TCP, with mTLS since v53; n-2 compatible from v54	the migration channel carries guest memory and is trusted input to the receiver. Use mTLS with Choir-issued certificates on a private network only, never on a public listener
snapshots	versioned per binary; File or UFFD memory backend	sparse files, UFFD restore, copy-on-write restore, an **offload daemon** protocol (stream through an external process that can encrypt)	both treat the snapshot as trusted. A tampered snapshot is guest-controlled host input, so keep snapshot directories writable only by the helper and the VM's own uid. CoW restore shares pages between forks (a Flush+Reload consideration for S9 forks of *different* tenants; same-tenant forks are fine)
nested virtualization	not exposed	exposed on x86 if the host allows it	keep `kvm_intel nested=0` unless a product needs it; a guest-controlled hypervisor inside the guest exercises a large, less-tested part of KVM

5.3 One contract, two backends

The backend seam proposed in the isolation-runtimes report (`Backend` with `Launch`, `Pause`, `Snapshot`, `Restore`, `OwnsProcess`, `Caps()`) should carry a `Confinement` value that every backend must honour:

```
// Confinement is what a VMM process may touch. Every backend must satisfy
// it; vmctl verifies it from /proc after launch and refuses to serve a VM
// whose VMM does not match.
type Confinement struct {
    UID, GID      uint32   // per-VM, from the reserved range
    JailRoot      string   // the only filesystem the VMM sees
    Files         []JailFile // host path -> jail path, mode, read-only
    NetNSPath     string   // per-VM network namespace (slice 2)
    TapFD         int      // for backends that accept fds (CH); -1 for FC
    CgroupPath    string   // the VM's cgroup v2 leaf
    CPUWeight     uint64
    MemoryMax     uint64
    PidsMax       uint64
    IOMax         []IOLimit
    RLimits       RLimits
    CoreSchedKey  string   // tenant (owner) id; "" = no core scheduling
    SpecCtrl      bool     // IBPB/STIBP + SSBD opt-in before exec
    InnerSandbox  InnerSandbox // backend-specific: FC seccomp level;
                               // CH seccomp=true, Landlock rules
}
```

- The **outer cage** (uid, jail, namespaces, cgroup, rlimits, speculation, core scheduling) is backend-neutral and owned by Choir's helper.
- The **inner sandbox** (seccomp, Landlock) is each VMM's own, configured at its strictest.
- A post-launch **verifier** reads `/proc/<pid>/{status,cgroup,ns/*,root}` and compares them with the `Confinement`. A mismatch kills the VMM and fails the boot closed. That turns the contract into something checked on every launch.
- The LT matrix runs the same assumed-breach probe against both backends.

6. Jailer prior art: what to reuse before building our own

Owner (about 17:50 UTC): "We would have to build the Cloud Hypervisor jailer ourselves. Is there open source code that we should review before we build it ourselves?" Yes. The code below was read from each project's `main` on 2026-10-14 (shallow clones; nothing was built or run). Claims the survey could not confirm are marked UNVERIFIED.

6.1 The candidates

project	licence	maturity	how it launches and confines a VMM	fit for one Choir jailer for both VMMs
Firecracker jailer (`src/jailer`)	Apache-2.0	ships with every Firecracker release (1.17.0, 2026-09-10)	the full cage described in section 1.4. Since **1.13.0** it no longer requires the binary name to contain "firecracker" (CHANGELOG; PR #5282). It still always prepends `--id <id> --start-time-us ...` to the binary's arguments, and closes every fd above 2	**Firecracker only.** Cloud Hypervisor rejects the prepended `--id`, and no tap fd survives. Read it as the reference for the order of setup steps
Cloud Hypervisor built-ins	Apache-2.0 / BSD-3	v54.0 (2026-10-14)	seccomp, Landlock and core scheduling inside the VMM; nothing outside it	**the inner layer for CH**, used together with an outer cage
firecracker-containerd runc jailer (`runtime/runc_jailer.go`)	Apache-2.0	maintained (last commit 2026-07); no GitHub releases	builds an OCI `config.json` and runs the VMM with `runc`: per-VM uid/gid (refuses 0), all namespaces, netns path, cgroup path, cpuset, **empty capability sets**, `noNewPrivileges`, and a device allowlist of `/dev/kvm` and `/dev/net/tun`; files placed by bind mount, copy or `mknod`	**the strongest prior art**: already VMM-agnostic in shape; only `Process.Args` is Firecracker-specific
Kata Containers (Go runtime and runtime-rs)	Apache-2.0	4.2.0 (2026-09-15)	Firecracker under the jailer **as uid 0** (both runtimes pass `--uid 0 --gid 0`). Cloud Hypervisor optionally "rootless": a throwaway `kata-NNNNN` user with `/dev/kvm`'s group, launched with `SysProcAttr.Credential`; taps sent to CH over the API socket with `SCM_RIGHTS`; a core-scheduling helper in Go (`schedcore_linux.go`)	**read for the CH-as-non-root and fd-passing code**; do not copy its Firecracker setup (root inside the jail)
flintlock (Liquid Metal)	MPL-2.0	v0.16.0 (October 2026)	plain `exec.Command` of either VMM **as root**; no jailer	only its provider interface (`core/ports/services.go`) is worth reading, as an abstraction shape
crosvm + minijail	BSD-3	active (ChromeOS, Android)	every virtio device in **its own jailed process** with its own seccomp policy, pid/user/net namespaces, empty capabilities and a tmpfs root	the gold standard for *device* isolation, which neither Firecracker nor CH does; the model is worth knowing, but the code is not reusable from Go
minijail (`libminijail`, `minijail0`)	BSD-3	tag linux-v2026.05.18	uid/gid, user/pid/net/mount namespaces, pivot_root, binds, rlimits, seccomp policy language, Landlock, preserved fds; joins an existing cgroup but sets no limits	a possible engine via its CLI; no Go binding (UNVERIFIED that none exists)
nsjail	Apache-2.0	3.6 (2026-03-18)	all namespaces, cgroup v2 `memory.max`/`pids.max`/`cpu.max`, rlimits, kafel seccomp, `pass_fd`, `iface_own` (moves an interface into the jail's netns), core scheduling	a capable CLI engine; **no `cpu.weight` or `io.max`**, both of which Choir's fractional-core plan needs
bubblewrap	LGPL-2.1+	v0.13.1	user-namespace sandbox; setuid mode removed in 0.12.0; no cgroups	**poor fit** (built for desktop apps)
systemd transient units (`systemd-run`, or D-Bus `StartTransientUnit`)	LGPL-2.1+	the init system already on Node B	one cgroup v2 leaf per unit (`CPUWeight`, `MemoryMax`, `TasksMax`, `IOWeight`, `IO*BandwidthMax`), `User=`, `SupplementaryGroups=kvm`, `RootDirectory=` + `BindPaths=`, `NetworkNamespacePath=`, `PrivatePIDs=`, `DevicePolicy=closed` + `DeviceAllow=/dev/kvm rw`, empty `CapabilityBoundingSet=`, `NoNewPrivileges=`, `RestrictNamespaces=`, `SystemCallFilter=`, `Limit*=`. No speculation or core-scheduling settings	**a strong engine** for the outer cage, with accounting and journald for free. Two caveats: `PrivateDevices=yes` hides `/dev/kvm`, so use `DevicePolicy=closed` + `DeviceAllow`; and `DynamicUser=` uids are recycled and cannot own a pre-created tap (the kernel needs the tap's owner to match the caller), so use **static per-VM uids**
apple/containerization	Apache-2.0	0.33.3	**has a Linux Cloud Hypervisor backend** (`CHVirtualMachineManager`): spawns `cloud-hypervisor` with a minimal environment and `setsid`, CH seccomp on; no uid drop, namespaces or cgroups	a precedent for a two-backend manager; nothing to reuse for confinement

project	licence	maturity	how it launches and confines a VMM	fit for one Choir jailer for both VMMs
firecracker-go-sdk (`jailer.go`)	Apache-2.0	maintained	Go argument builder for the jailer; hard-links kernel and drives into the chroot; no `--resource-limit` or `--new-pid-ns`	useful if Choir drives the Firecracker jailer from Go
microvm.nix host module	MIT	active	`microvm@<name>` systemd template units with one shared `microvm` user; taps created `user microvm`	pattern only (one user for all VMs is weaker than per-VM)
E2B infra	Apache-2.0	active	`unshare -m` + `ip netns exec`, Firecracker as root, no jailer	counter-example: what not to copy

Fly.io writes that it uses the jailer, but its manager is closed source (UNVERIFIED). kraftkit, vmm-reference and Lima were not checked.

One lesson from CVE-2026-1386 (the jailer followed a symlink while copying into a jail directory a local user could write): every jail path and its parents must be root-owned and writable by no one else, and the launcher must open with `O_NOFOLLOW` and check what it opened.

6.2 Recommendation: wrap, do not build from scratch

- **Do not adopt the Firecracker jailer for both VMMs.** It is scoped to Firecracker by design. **Do use it for Firecracker now** (slice J1). It is the shortest path to a privilege drop on the backend that runs today, and it is the configuration upstream supports.
- **Build one thin Choir cage in Go that wraps an existing engine**, modelled on firecracker-containerd's runc jailer:
    1. *Privileged steps in the helper (root).* Create the netns and the tap inside it, already up, MAC and MTU set, owned by the VM's static uid. Prepare a root-owned 0700 jail directory. Place the VM's files.
    2. *Outer cage by systemd transient unit* (`StartTransientUnit` over D-Bus, unit `choir-vm-<id>.service` in `choir-vms.slice`) with the properties in the table above. This gives Choir the cgroup v2 leaf with `CPUWeight` and `IO*` that nsjail lacks, plus accounting, journald capture and `systemctl` inspection. nsjail or minijail0 is the fallback engine if a host lacks systemd (the desktop).
    3. *Inner sandbox per VMM.* Firecracker uses its seccomp filters. Cloud Hypervisor uses `--seccomp true`, `--landlock` with explicit rules, `--cpus core_scheduling=vm`, a tap by fd, and `image_type=raw`.
    4. *Speculation control and per-tenant core-scheduling cookies* are set by the helper in a pre-exec step, because no engine sets them. Kata's `schedcore_linux.go` is a ready Go pattern.
    5. *The post-launch verifier* from section 5.3.
- Once the cage exists, Firecracker can move from its jailer to the same cage (an `ExecStart` without the jailer), so that both backends share one verified contract. Keep the jailer path until the cage has passed LT's assumed-breach probe on both.
- **Verify before building**: snapshot, restore and balloon under `RootDirectory=` + `BindPaths=` for both VMMs, and whether Cloud Hypervisor needs `/proc` or `/sys` inside its root (UNVERIFIED).

6.3 Read first, in this order

1. firecracker-containerd: `runtime/runc_jailer.go`, `runtime/firecracker-runc-config.json.example`, `runtime/jailer.go`
2. Firecracker jailer: `src/jailer/src/env.rs` (`run`, `copy_exec_to_chroot`, `mknod_and_own_dev`), then `chroot.rs` and `cgroup.rs`
3. `docs/jailer.md`, `docs/prod-host-setup.md`, GHSA-36j2-f825-qvgc (CVE-2026-1386)
4. Kata: `virtcontainers/clh.go` (SCM_RIGHTS taps, `Credential`), `containerd-shim-v2/create.go`, `pkg/utils/schedcore_linux.go`, `docs/how-to/how-to-run-rootless-vmm.md`
5. Cloud Hypervisor: `docs/threat-model.md`, `docs/landlock.md`, `net_util/src/open_tap.rs`, `release-notes.md` (v52 to v54)

6. systemd.exec(5) (DynamicUser, PrivateDevices, NetworkNamespacePath, PrivatePIDs) and systemd.resource-control(5) (CPUWeight, MemoryMax, DeviceAllow)
7. crosvm: `jail/src/helpers.rs`
8. nsjail `config.proto` and `cgroup2.cc`; minijail `libminijail.h`
9. flintlock `core/ports/services.go`; microvm.nix `nixos-modules/host/default.nix`

7. The plan: now, this week, this month

Every item names its mutation class, effort (S: under a day; M: two to five days; L: one to three weeks), the risk that it breaks something, how to prove it on staging, how to roll it back, and the station it belongs to: **S1** (security floor), **LT** (red team), **SP** (second host and production hardening), **LR** (lifecycle and restart).

7.1 Now (today and tomorrow)

#	item	class	effort	breakage risk	staging proof	rollback	station
N1	**Deploy `fix/tap-deny-guest-range`** (816077e83) after adding the inbound rule `-o vm-+ -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT` (then DROP)	red	S	medium: a wrong chain cuts guest egress or host-to-guest paths; the branch guards the rebuild with a DROP, so failure is closed	`iptables -S CHOIR-GUEST-FWD`; 0 stale `vm-vm-*` rules after start; rerun the S1a refusal matrix (5/5 refusals, 5/5 legitimate flows); an allocation forced to `10.201.x` cannot reach a `10.200.x` guest; counters on the new DROPs	`git revert` and redeploy, then the manual `iptables` deletes listed in the commit	S1
N2	**Kill the stray `python3 -m http.server 19090`** (root, `/tmp`, since 30 September) and find who started it	n/a (operator action)	S	none	`ss -ltnp` shows no `:19090`	none needed	S1
N3	**File problem docs** (problem-documentation-first) for F2 (loopback is the credential), F3 (host services as root; `@privileged` re-added), F4 (desk workers as guest root), F8 (inbound forward), F10 (maild recipient; escrow transport), F12 (snapshots and disks in plaintext), F13 (CI as root)	green	S	none	the docs exist and cite this review	n/a	S1
N4	**KSM off and `nested=0`** in `nix/node-b.nix` (`hardware.ksm.enable = false` or `/sys/kernel/mm/ksm/run=0`; `boot.extraModprobeConfig = "options kvm_intel nested=0"`)	red (host kernel config)	S	low: KSM merges nothing today; Firecracker does not use nested virtualization (needs a reboot or module reload for `nested`)	`cat /sys/kernel/mm/ksm/run` = 0; `/sys/module/kvm_intel/parameters/nested` = N after the next reboot	revert the Nix change	SP

7.2 This week: the VMM privilege drop, sliced

J1 — the jailer, slice 1 (the first and most important item). Run every Firecracker under the jailer, in the host network namespace for now.

- **Scope.** A `Confinement` struct in vmmanager (uid, gid, jail root, cgroup path, netns path, rlimits); per-VM uid/gid allocation from the host-port ordinal; a jail directory per VM under `/var/lib/go-choir/jail/firecracker/<vm-id>/root/` with hard links of `data.img`, `credential.img`, kernel, initrd and the store image; chown of the per-VM files; launch through `jailer --id <vm-id> --exec-file <firecracker> --uid --gid --chroot-base-dir /var/lib/go-choir/jail --new-pid-ns --cgroup-version 2 --parent-cgroup choir-vms.slice --cgroup memory.max=... --cgroup cpu.weight=... --resource-limit no-file=2048 -- --api-sock /run/fc.sock --enable-pci --config-file /fc-config.json`; Firecracker

config paths rewritten relative to the jail; snapshot paths relative to the jail; process tracking by pidfd and cgroup instead of cmdline scanning; `cmd.Env` emptied; `PR_SET_SPECULATION_CTRL` (indirect branch and store bypass) set before exec; a systemd slice `choir-vms.slice` with `Delegate=yes`; an `OUTPUT -m owner --uid-owner 900000-904095 -j DROP` rule so VMM uids hold no IP connectivity; `CAP_SYS_PTRACE` removed from vmctl's bounding set.

- **Class** red (vmctl, VM lifecycle, snapshot validity). **Effort** M (three to five days including staging). **Breakage risk** medium-high: snapshot identity changes once (every hibernated computer cold-boots at the cutover; schedule it with a planned-restart marker or when computers are idle), console capture, reattach after a vmctl restart, and the balloon socket all touch paths.
- **Order inside the slice**: (a) the `Confinement` struct and jail preparation behind a flag, Firecracker still launched directly (no behaviour change); (b) jailer launch for *new* computers on staging only, behind `VM_CONFINEMENT=jailer`; (c) hibernate, wake, crash, vmctl restart and deploy-refresh exercised on a disposable computer; (d) migrate existing computers at their next cold boot; (e) remove the direct-launch path.
- **Staging proof** (record as `docs/evidence/vmm-confinement-<date>.json`): for each VMM pid, `/proc/<pid>/status` shows `Uid` in the reserved range, `CapEff: 0000000000000000`, `NoNewPrivs: 1`, `Seccomp: 2`; `readlink /proc/<pid>/root` is the jail; `ns/mnt` and `ns/pid` differ from init and from every other VMM; `/proc/<pid>/cgroup` is its own leaf with the expected `memory.max`; `SpeculationIndirectBranch: conditional force disabled` (or `disabled`) and `Speculation_Store_Bypass: thread force mitigated`. **Assumed-breach probe**: `nsenter` into the VMM's namespaces with its uid (`setpriv --reuid --regid --clear-groups`, on a disposable computer) and attempt to read another computer's `data.img`, `gateway-provider.env` and `auth-signing/ed25519-key`, to connect to `127.0.0.1:8083`, and to list `/proc`; every attempt fails. Plus the existing lifecycle proofs: boot, hibernate, wake (wake latency within the density benchmark's bounds), crash-restart closes work as before.
- **Rollback**: the flag (`VM_CONFINEMENT=direct`) for new launches; `git revert` for the code; computers launched under the jailer keep running under it until their next boot (KillMode=process).

#	item	class	effort	breakage risk	staging proof	rollback	station
W1	**J1** above	red	M	medium-high	above	above	S1 → LT baseline
W2	**Authenticate vmctl's internal API**: unix socket with `SO_PEERCRED` allowlist for lifecycle routes; TCP keeps only guest-scoped ComputerVersion routes; drop `192.0.2.*` from production; pprof behind a flag	red	M	medium: every host caller of vmctl moves to the socket	as a non-allowlisted uid (`setpriv` from a disposable unit), `curl` to `/internal/*` stop/remove is refused; proxy and gateway paths still green; S1a matrix still green	revert; the TCP listener is still there	S1
W3	**Dolt password** via `LoadCredential=`; DSNs updated	red	S	medium: every Dolt client must get the credential	`mysql -u root -h 127.0.0.1` without password refused; all services healthy	revert	S1
W4	**Fix `SystemCallFilter`** (drop `@privileged`) and narrow `ReadWritePaths` for proxy (auth public key only) and gateway	red	S	medium: a service that used a privileged syscall fails at start (find out with `SystemCallLog=` first)	`systemctl show -p SystemCallFilter` lacks `bpf`, `chroot`; services healthy; proxy cannot open `auth-signing/ed25519-key`	revert	S1
W5	**Generalize the Jev pin to every gateway route** (bearer valid only from its computer's current guest IP)	red (gateway)	S-M	low-medium: a re-addressed computer whose pin did not move gets 403 (the ensure path already handles it for Jev)	a token copied from computer A is refused from computer B; normal inference green after a cold boot	revert	S1

#	item	class	effort	breakage risk	staging proof	rollback	station
W6	**maild recipient from the registry, not the header**; escrow operator routes loopback-only	red	S	low	a guest-originated alert with a forged `X-Authenticated-Email` goes to the owner's address of record; operator route from a tap address refused	revert	S1
W7	**sshd closed to guests** (`-i vm-+ --dport 22 -j DROP`) and **egress floor** (drop SMTP 25 and private and metadata ranges from guests)	red	S	low	from a guest, `nc -z <tap .1> 22` times out; port 25 out times out; normal egress works	revert	S1

7.3 This month

#	item	class	effort	breakage risk	staging proof	rollback	station
M1	**Jailer slice 2: per-VM network namespace** with the tap inside and a veth to the host; firewall rules move to the veth; drop the uid-owner OUTPUT rule	red	M	medium-high (all guest networking)	`/proc/<pid>/ns/net` per VM differs from init; S1a matrix green; assumed-breach probe sees no host interfaces	flag back to host netns	S1 / LT
M2	**vmctl privilege split**: `choir-vmhelper` (root, narrow typed API) and vmctl as an unprivileged user	red	L	medium	vmctl `Uid` non-zero, `CapEff 0`; helper refuses requests from other uids; full lifecycle proof	run both in one process (the split is a package boundary first)	S1 / SP
M3	**Host services to their own users** (`DynamicUser` or static), empty capability sets, `IPAddressDeny`, `LoadCredential=` for secrets; SearXNG and goproxy unable to reach other loopback ports	red	M-L	medium (state ownership migrations)	`systemctl show` per unit; from the searx uid, `127.0.0.1:8083` and `:13306` unreachable	per-unit revert	S1 / SP
M4	**Desk cell workers in a capsule-lite**; S1 non-root guest runtime; `pdftotext` and Obscura sandboxed; Obscura address guard; `clone3` → `ENOSYS` in the capsule filter	red	L	medium-high (every desk turn)	the S1 acceptance items 3 to 5 on staging; a cell that tries `os` import is refused (already) and a worker process shows non-zero uid, seccomp 2	revert per item	S1
M5	**Whole-guest egress default-deny** through a host-side allowlisting proxy (the S1 item)	red	M-L	high (anything a guest fetches)	S1 acceptance item 2	allow-all policy flag	S1
M6	**Snapshot volume with a per-boot ephemeral key**; `data.img` 0600 owned by the VM uid; LUKS planned for the EPYC	red	M	medium (snapshot path)	`dmsetup table` shows the crypt target; a host reboot cold-boots hibernated computers without data loss	snapshots back on the plain volume	SP / LR
M7	**Firecracker 1.17.0** with the jailer (snapshot format bump: pair with J1's cutover so computers pay one cold boot, not two)	red	S-M	medium	version in `/proc/<pid>/cmdline` path; lifecycle proof	pin back to 1.15.1 (never below, while PCI is on)	S3 / LR
M8	**CI deploy without root**: a deploy user that can only trigger a root-owned `systemd` deploy unit with a fixed command; Actions pinned by commit SHA	red (deployment routing)	M	medium (deploys)	the deploy key's `authorized_keys` line has `command=` and `restrict`; a deploy runs green	restore the root key line	SP

#	item	class	effort	breakage risk	staging proof	rollback	station
M9	**Core scheduling per tenant** in the helper, measured (`force_idle_usec`, throughput, p99) on the EPYC or the LT host before strangers are admitted	red	M	low (performance only)	`/proc/<pid>/sched` core cookie set; two tenants' vCPUs never co-run on siblings (perf sched trace)	flag	SP / LT
M10	**Backend seam with the `Confinement` contract**, Firecracker only, then Cloud Hypervisor on the LT host (section 5)	red	L	low (refactor first)	existing vmmanager tests and one staging lifecycle	revert the refactor commit	SP / LT

8. What the red-team station LT should test first

Rules of engagement. Use disposable hosts only. That means a wipeable bare-metal box of the same CPU family as the target, reimaged from the NixOS config between campaigns. Never use Node B, production data or real provider keys. Use a stub gateway and stub keys, and send the host's egress through an allowlisted sink. Targets are synthetic computers minted from the production image. Every finding becomes a problem doc before any fix. The detailed experiment design lives in the isolation-runtimes report §7 (E1 to E8). This section only sets the order and the pass conditions that the work in section 7 must meet.

Run each area against two targets: today's shape (**T1**: Firecracker as root, no jailer) and the confined shape (**T2**: J1 plus M1). The difference between the two measures what the hardening bought.

order	area	question it answers	pass condition on T2
1	**Assumed VMM breach** (E1)	With the VMM's own identity, namespaces, cgroup and seccomp filter, what can a process reach?	Only its own jail, tap and `/dev/kvm`. No other computer's files, no host secrets, no host loopback services, no other processes, no firewall changes. Its cgroup bounds memory and tasks.
2	**Egress and host reachability** (E4)	From a guest, a capsule and a cell worker, which destinations answer? This covers sibling guests, host ports, sshd, the provider's metadata and private ranges, and the internet.	Only allowlisted destinations and the declared host service ports. Each host service accepts only the caller's own scoped authority.
3	**Capsule and cell containment** (E3)	Does model-written code stay inside its boundary? Can it read the gateway token or reach guest root?	No token is recoverable and no guest root is reached. Compare today's namespace capsule with a gVisor capsule.
4	**Resource isolation** (E5)	Can one computer degrade its neighbours?	Per-VM cgroups bound each computer. Neighbours' wake latency and pressure stay inside the density benchmark's safety stops.
5	**Side channels** (E6)	How much leaks between tenants under each SMT and core-scheduling setting?	A measured leakage figure for each configuration. These figures feed the per-host SMT policy in section 4.7.
6	**Snapshot and management-plane integrity** (E7, E8)	Is a tampered snapshot refused? Can a guest reach a VMM API socket or a vmctl route it does not own?	Tampering is refused or contained. The sockets and routes answer only their intended callers.

Area 1 needs no exploit and no new runtime, so it can run as soon as the LT host exists. It is also the acceptance test for J1. Areas 2 and 4 reuse the staging proofs from section 7.

9. Glossary

term	meaning
Blast radius	What an attacker controls after one boundary fails.
Bounding set (capabilities)	The ceiling of capabilities a process and its children can ever hold.
Capability (Linux)	One of about 40 pieces of root's power, such as `CAP_NET_ADMIN` (networking and firewall), `CAP_SYS_PTRACE` (inspecting other processes) and `CAP_DAC_OVERRIDE` (ignoring file permissions).

term	meaning
Capsule	Choir's guest-local sandbox for engineering's model-written processes: namespaces, seccomp, Landlock, cgroup and an egress broker.
Cell	One model-written Go fragment run by a desk in a Yaegi worker.
cgroup v2	The kernel's resource-control tree. A leaf's `cpu.weight`, `memory.max`, `io.max` and `pids.max` bound what its processes may use.
chroot / pivot_root	Making a directory the process's `/`, so nothing outside it exists for that process.
Core scheduling	A scheduler feature that lets two threads share a physical core at the same time only if they carry the same cookie.
Defence in depth	Layering independent boundaries so that one failure is not enough.
Device model	The set of virtual devices a VMM emulates. It is the main attack surface a guest can reach.
DNAT / MASQUERADE	iptables address rewriting: destination NAT redirects traffic to another address, and masquerade rewrites the source to the host's address on the way out.
Escape (VM)	Guest-controlled input that leads to code execution outside the guest: usually in the VMM process, more rarely in the host kernel.
IBPB / STIBP / SSBD	CPU barriers against Spectre-class leaks: flushing branch predictors between contexts, isolating sibling threads' predictors, and disabling speculative store bypass.
Jailer	Firecracker's launcher. It puts each VMM in its own uid/gid, chroot, namespaces and cgroup before running it.
KSM	Kernel samepage merging, which deduplicates identical memory pages across processes. It is a cross-tenant side channel for VMs.
KVM	The Linux kernel's hardware virtualization module, reached through `/dev/kvm`.
L1TF / MDS	Intel CPU flaws that let one hyperthread read data its sibling touched. The kernel cannot mitigate them while SMT is on.
Landlock	A Linux security module that lets a process restrict its own file access, without root.
Namespace (Linux)	A per-process view of one kernel resource: mounts, PIDs, network, users, UTS, IPC or cgroups.
NoNewPrivs	A process flag that forbids gaining privileges through exec (setuid binaries, file capabilities).
pidfd	A file descriptor that refers to one process. It is safer than a PID number, which can be reused.
route_localnet	A sysctl that lets packets addressed to 127.0.0.0/8 arrive on a non-loopback interface.
Seccomp	A kernel filter on the system calls a process (or thread) may make. Once installed, the process cannot remove it.
Side channel	Leaking data through shared hardware timing rather than through a software bug.
SMT	Simultaneous multithreading (Hyper-Threading): two hardware threads sharing one physical core.
SO_PEERCRED	A unix-socket option that tells a server the connecting process's uid, gid and pid. The client cannot forge it.
Tap	A virtual network interface that a VMM reads and writes as a file. Each Choir guest has one.
UFFD (userfaultfd)	A kernel mechanism that lets a process serve page faults. It is used for lazy snapshot restore.
virtio / virtqueue	The standard paravirtual device interface: requests in shared-memory rings, processed by the VMM.
VMM	Virtual machine monitor: the user-space process (Firecracker, Cloud Hypervisor) that emulates a VM's devices.
vsock	A host-guest socket transport whose peer address (CID) is set by the VMM, so the guest cannot spoof it.

Residuals, confidence and sources

- **Read live today:** the state, namespaces and cgroups of the VMM and vmctl processes; `systemctl show` for vmctl and the other units; file ownership and modes under `/var/lib/go-choir` (names only); `iptables -S` for the filter, nat and mangle tables; listeners; sysctls; the CPU vulnerability files and KVM parameters; the effective sshd configuration; block devices. No secret value was read.
- **Read in code but not proven on staging:** everything about cells, capsules and the guest runtime (sections 3.1 and 3.2), the authentication rules of the host services (section 3.5), and snapshot handling (section 3.8). Each finding cites its path. Where a finding is reasoning rather than reading, it says *inference*.

- **Open questions for LT:** how far a Firecracker thread under seccomp can reach after a device-model bug; whether hosts on Node B's provider segment can forward traffic in to guests (F8); what the 2,417 packets that hit the FORWARD policy are; whether Web Lens can be used for SSRF; how the desk file jail handles symlinks.
- **Not covered:** the frontend and the auth flows beyond the proxy's header handling; Caddy's configuration; the SP standby and backups (neither exists yet to review); the macOS desktop runtime.
- **Prior documents this review builds on:** firecracker-runs-as-root-without-jailer, tap-deny-rule-covers-one-of-sixteen-guest-subnets, isolation-runtimes, computer-cpu-efficiency §5, vm-density-limits, self-hosting-and-desktop, S1 security floor, S1 non-root runtime remainder, capsule Go module proxy design, computer ontology, Choir doctrine (I18, I19).
- **Upstream sources:**
  - Firecracker: the jailer, production host setup and advisories.
  - Cloud Hypervisor: the threat model, advisories and release notes.
  - Linux kernel documentation: core scheduling, L1TF, MDS and speculation control.
  - The sources listed in section 6.